Problem Set 2 — Industrial Organization II

Setup

In [1]:
```
import numpy as np
import pandas as pd
import pyblp
import statsmodels.api as sm
from linearmodels.iv import IV2SLS
import warnings
warnings.filterwarnings('ignore')

pyblp.options.verbose = False
np.random.seed(42)

T, J = 600, 4
```

Load Data

In [2]:
```
data = pd.read_csv('pset2_data.csv').drop(columns='Unnamed: 0')
data = data.sort_values(['market_ids', 'firm_ids']).reset_index(drop=True)

print(data.shape)
data.head()
```

```
(2400, 8)
```

Out[2]:

	market_ids	firm_ids	prices	shares	satellite	wired	x	w
0	0	0	2.207442	0.016287	1.0	0.0	0.616935	0.564690
1	0	1	4.109471	0.657101	1.0	0.0	0.202908	0.839746
2	0	2	3.213095	0.052683	0.0	1.0	0.288809	0.376884
3	0	3	2.866005	0.013843	0.0	1.0	0.445060	0.499676
4	1	0	3.003386	0.723136	1.0	0.0	0.547227	0.081302

In [3]:
```
data['outside_share'] = 1 - data.groupby('market_ids')['shares'].transform('sum
data['ln_sj_s0'] = np.log(data['shares']) - np.log(data['outside_share'])

# within-nest shares
for nest in ['satellite', 'wired']:
    mask = data[nest] == 1
    g_sum = data[mask].groupby('market_ids')['shares'].transform('sum')
    data.loc[mask, 'within_share'] = data.loc[mask, 'shares'] / g_sum

data['ln_within'] = np.log(data['within_share'])
data['sat_ln_within'] = data['satellite'] * data['ln_within']
data['wire_ln_within'] = data['wired'] * data['ln_within']

# sum of x and w over competing goods in same market
for var in ['x', 'w']:
```

```
    data[f'{var}_other'] = data.groupby('market_ids')[var].transform('sum') - (

# same-nest competitor instruments
for nest in ['satellite', 'wired']:
    for var in ['x', 'w']:
        mask = data[nest] == 1
        nest_sum = data[mask].groupby('market_ids')[var].transform('sum')
        col = f'{var}_other_{nest}'
        data.loc[mask, col] = nest_sum - data.loc[mask, var].values
        data[col] = data[col].fillna(0)

data[['shares','outside_share','prices','x','w']].describe().round(3)
```

Out[3]:

	shares	outside_share	prices	x	w
count	2400.000	2400.000	2400.000	2400.000	2400.000
mean	0.183	0.269	2.675	0.498	0.492
std	0.180	0.152	0.694	0.289	0.289
min	0.000	0.012	1.162	0.000	0.000
25%	0.032	0.155	2.198	0.249	0.245
50%	0.118	0.237	2.568	0.490	0.496
75%	0.295	0.348	3.042	0.748	0.748
max	0.781	0.852	7.182	1.000	0.999

Display function

In [4]:

```
def matrix_df(mat):
    labs = [f'J{j+1}' for j in range(J)]
    return pd.DataFrame(mat.round(4), index=labs, columns=labs)
```

Q1: Logit OLS

In [5]:

```
X = data[['x', 'satellite', 'wired', 'prices']].values
y = data['ln_sj_s0'].values

ols = sm.OLS(y, X).fit(cov_type='HC1')
ols_params = ols.params

pd.DataFrame({
    'param': ['beta1', 'beta_sat', 'beta_wire', 'alpha'],
    'true':  [1.0, '~4', '~4', -2.0],
    'ols':   ols.params.round(4),
    'se':    ols.HC1_se.round(4)
})
```

Out[5]:

	param	true	ols	se
0	beta1	1.0	0.9419	0.1252
1	beta_sat	~4	1.1988	0.1721
2	beta_wire	~4	1.2108	0.1741
3	alpha	-2.0	-1.0218	0.0636

Q2: Logit 2SLS

In [6]:

```
iv = IV2SLS(
    dependent=data['ln_sj_s0'],
    exog=data[['x', 'satellite', 'wired']],
    endog=data[['prices']],
    instruments=data[['w', 'x_other', 'w_other']]
).fit(cov_type='unadjusted')

pd.DataFrame({
    'param':    ['beta1', 'beta_sat', 'beta_wire', 'alpha'],
    'true':     [1.0, '~4', '~4', -2.0],
    'ols':      ols_params.round(4),
    'tsls':     iv.params.values.round(4),
    'tsls_se':  iv.std_errors.values.round(4)
})
```

Out[6]:

	param	true	ols	tsls	tsls_se
0	beta1	1.0	0.9419	1.0623	0.1410
1	beta_sat	~4	1.1988	4.5716	0.9966
2	beta_wire	~4	1.2108	4.5523	0.9874
3	alpha	-2.0	-1.0218	-2.2990	0.3759

Q3: Nested Logit 2SLS

In [7]:

```
ivs = data[['w', 'x_other', 'w_other',
            'x_other_satellite', 'x_other_wired',
            'w_other_satellite', 'w_other_wired']]

nl = IV2SLS(
    dependent=data['ln_sj_s0'],
    exog=data[['x', 'satellite', 'wired']],
    endog=data[['prices', 'sat_ln_within', 'wire_ln_within']],
    instruments=ivs
).fit(cov_type='unadjusted')

alpha_nl    = nl.params['prices']
beta1_nl    = nl.params['x']
sigma_sat   = nl.params['sat_ln_within']
sigma_wire  = nl.params['wire_ln_within']

pd.DataFrame({
    'param': ['beta1', 'beta_sat', 'beta_wire', 'alpha', 'sigma_sat', 'sigma_w
```

```
    'true':  [1.0, '~4', '~4', -2.0, '--', '--'],
    'est':   [round(nl.params[k], 4) for k in
              ['x','satellite','wired','prices','sat_ln_within','wire_ln_withi
    'se':    [round(nl.std_errors[k], 4) for k in
              ['x','satellite','wired','prices','sat_ln_within','wire_ln_withi
})
```

Out[7]:

	param	true	est	se
0	beta1	1.0	0.8848	0.1612
1	beta_sat	~4	4.1862	0.9119
2	beta_wire	~4	4.1951	0.9350
3	alpha	-2.0	-1.9937	0.3862
4	sigma_sat	--	0.2761	0.2776
5	sigma_wire	--	0.2979	0.2100

Q4: Nested Logit Elasticities and Diversion Ratios

In [8]:

```
def nl_elas_div(df, alpha, sig_sat, sig_wire):
    elas_all = np.zeros((T, J, J))
    div_all  = np.zeros((T, J, J))

    for t_i, t in enumerate(df['market_ids'].unique()):
        mkt = df[df['market_ids'] == t].sort_values('firm_ids').reset_index(dr
        s   = mkt['shares'].values
        p   = mkt['prices'].values
        sw  = mkt['within_share'].values
        sat = mkt['satellite'].values.astype(int)
        sig = np.where(sat == 1, sig_sat, sig_wire)

        for j in range(J):
            sj, swj, pj, sigj = s[j], sw[j], p[j], sig[j]
            ds_own = (alpha/(1-sigj)) * sj * (1 - sigj*swj - (1-sigj)*sj)
            elas_all[t_i, j, j] = ds_own * pj / sj

            for k in range(J):
                if k == j: continue
                sk, swk, pk, sigk = s[k], sw[k], p[k], sig[k]
                same = (sat[j] == sat[k])

                if same:
                    ds_j_dpk = -(alpha/(1-sigk)) * sj * (sigk*swk + (1-sigk)*sk
                else:
                    ds_j_dpk = -alpha * sj * sk
                elas_all[t_i, j, k] = ds_j_dpk * pk / sj

                if same:
                    ds_k_dpj = -(alpha/(1-sigj)) * sk * (sigj*swj + (1-sigj)*sj
                else:
                    ds_k_dpj = -alpha * sk * sj
                div_all[t_i, j, k] = ds_k_dpj / (-ds_own)

    return elas_all.mean(axis=0), div_all.mean(axis=0)
```

In [9]:
```
nl_elas, nl_div = nl_elas_div(data, alpha_nl, sigma_sat, sigma_wire)

true_own_elas = pd.read_csv('true_ownprice_elasticities.csv',
                            header=0, index_col=0).values.flatten().astype(flo
true_div = pd.read_csv('true_diversionratio.csv',
                       header=0, index_col=0).values.astype(float)

# own-price elasticity comparison
print('Own-price elasticities:')
display(pd.DataFrame({
    'product': ['J1 (sat)', 'J2 (sat)', 'J3 (wire)', 'J4 (wire)'],
    'nl_est':  np.diag(nl_elas).round(4),
    'true':    true_own_elas.round(4),
    'diff':    (np.diag(nl_elas) - true_own_elas).round(4)
}))

print('\nEstimated diversion ratios (NL):')
display(matrix_df(nl_div))

print('\nTrue diversion ratios:')
display(matrix_df(true_div))
```

```
Own-price elasticities:
```

	product	nl_est	true	diff
0	J1 (sat)	-5.3539	-4.2500	-1.1040
1	J2 (sat)	-5.5501	-4.3714	-1.1787
2	J3 (wire)	-5.5184	-4.2711	-1.2473
3	J4 (wire)	-5.5120	-4.2576	-1.2544

```
Estimated diversion ratios (NL):
```

	J1	J2	J3	J4
J1	0.0000	0.3334	0.1865	0.1943
J2	0.3386	0.0000	0.1856	0.1861
J3	0.1839	0.1827	0.0000	0.3526
J4	0.1868	0.1771	0.3508	0.0000

```
True diversion ratios:
```

	J1	J2	J3	J4
J1	0.3338	0.2170	0.2204	0.2288
J2	0.2194	0.3370	0.2216	0.2220
J3	0.2193	0.2185	0.3350	0.2272
J4	0.2246	0.2150	0.2232	0.3371

Q5: Correctly Specified RC Logit

In [10]:
```
pd_blp = data[['market_ids','firm_ids','shares','prices',
               'x','satellite','wired','w']].copy()
```

```
X1 = pyblp.Formulation('0 + x + satellite + wired + prices')
X2 = pyblp.Formulation('0 + satellite + wired')
X3 = pyblp.Formulation('0 + 1 + w')

# BLP demand IVs on x only (sat+wire are collinear so drop them)
d_iv = pyblp.build_blp_instruments(pyblp.Formulation('0 + x'), pd_blp)
d_iv = d_iv[:, np.std(d_iv, axis=0) > 1e-12]  # drop zero-variance cols
for i in range(d_iv.shape[1]):
    pd_blp[f'demand_instruments{i}'] = d_iv[:, i]
pd_blp[f'demand_instruments{d_iv.shape[1]}'] = pd_blp['w'].values

# supply IVs
s_iv = pyblp.build_blp_instruments(pyblp.Formulation('0 + w'), pd_blp)
for i in range(s_iv.shape[1]):
    pd_blp[f'supply_instruments{i}'] = s_iv[:, i]
pd_blp[f'supply_instruments{s_iv.shape[1]}'] = pd_blp['x'].values

np.random.seed(0)
integration = pyblp.Integration('halton', size=500)
sigma0 = np.diag([0.5, 0.5])

pd_blp.head(4)
```

Out[10]:

	market_ids	firm_ids	shares	prices	x	satellite	wired	w	demand_ins
0	0	0	0.016287	2.207442	0.616935	1.0	0.0	0.564690	
1	0	1	0.657101	4.109471	0.202908	1.0	0.0	0.839746	
2	0	2	0.052683	3.213095	0.288809	0.0	1.0	0.376884	
3	0	3	0.013843	2.866005	0.445060	0.0	1.0	0.499676	

In [11]:

```
# (a) Demand-only, BLP instruments
pd_demand = pd_blp.drop(
    columns=[c for c in pd_blp.columns if 'supply_instruments' in c]
)

problem_d = pyblp.Problem(
    product_formulations=(X1, X2),
    product_data=pd_demand,
    integration=integration,
)
print(problem_d)

res_demand = problem_d.solve(
    sigma=sigma0,
    optimization=pyblp.Optimization('l-bfgs-b', {'gtol': 1e-8}),
    iteration=pyblp.Iteration('squarem'),
    W_type='unadjusted',
    se_type='unadjusted',
)
print(res_demand)
```

```
Dimensions:
========================================
 T    N     F     I      K1    K2    MD
---  ----  ---  ------  ----  ----  ----
600  2400   4   300000   4     2     5
========================================

Formulations:
=================================================================
       Column Indices:             0          1        2      3
-----------------------------  ---------  ---------  -----  ------
 X1: Linear Characteristics        x      satellite  wired  prices
X2: Nonlinear Characteristics  satellite    wired
=================================================================
Problem Results Summary:
=================================================================================
=====================================
GMM     Objective      Projected    Reduced Hessian  Reduced Hessian  Clipped
Weighting Matrix  Covariance Matrix
Step      Value      Gradient Norm  Min Eigenvalue   Max Eigenvalue   Shares
Condition Number  Condition Number
----  -------------  -------------  ---------------  ---------------  -------
----------------  -----------------
 2    +5.794689E-17  +2.418965E-09   +4.168947E-05    +5.565178E-02      0
+1.366073E+02      +3.725041E+16
=================================================================================
=====================================

Cumulative Statistics:
==============================================================================
Computation  Optimizer  Optimization   Objective   Fixed Point  Contraction
   Time      Converged   Iterations   Evaluations  Iterations   Evaluations
-----------  ---------  ------------  -----------  -----------  -----------
 00:00:22       Yes          5            11          46197       143209
==============================================================================

Nonlinear Coefficient Estimates (Unadjusted SEs in Parentheses):
=============================================
 Sigma:       satellite          wired
---------  ---------------  ---------------
satellite   +2.821025E-01
           (+1.926128E+00)

  wired     +0.000000E+00    +3.402452E+00
                            (+5.679761E+00)
=============================================

Beta Estimates (Unadjusted SEs in Parentheses):
=================================================================
       x            satellite          wired           prices
---------------  ---------------  ---------------  ---------------
 +1.310431E+00    +6.115517E+00    +5.188147E+00    -2.920280E+00
(+6.612045E-01)  (+4.196925E+00)  (+2.334492E+00)  (+1.684955E+00)
=================================================================
```

In [12]:

```
# rebuild product data with manual IVs for joint estimation
pd_blp = data[['market_ids','firm_ids','shares','prices',
               'x','satellite','wired','w']].copy()

pd_blp['demand_instruments0'] = data['w']
```

```
pd_blp['demand_instruments1'] = data['x_other']
pd_blp['demand_instruments2'] = data['w_other']
pd_blp['demand_instruments3'] = data['x_other_satellite']
pd_blp['demand_instruments4'] = data['x_other_wired']

pd_blp['supply_instruments0'] = data['x']
pd_blp['supply_instruments1'] = data['x_other']
pd_blp['supply_instruments2'] = data['x_other_satellite']
pd_blp['supply_instruments3'] = data['x_other_wired']

problem_j = pyblp.Problem((X1, X2, X3), pd_blp, integration=integration)

res_joint = problem_j.solve(
    sigma=sigma0,
    beta=np.array([[1.0],[4.5],[4.5],[-2.0]]),
    optimization=pyblp.Optimization('l-bfgs-b', {'gtol': 1e-8}),
    iteration=pyblp.Iteration('squarem'),
    W_type='unadjusted', se_type='unadjusted'
)
print(res_joint)
```

```
Problem Results Summary:
================================================================================
====================================
GMM     Objective      Projected    Reduced Hessian  Reduced Hessian  Clipped
Weighting Matrix  Covariance Matrix
Step      Value      Gradient Norm  Min Eigenvalue   Max Eigenvalue   Shares
Condition Number  Condition Number
----  -------------  -------------  ---------------  ---------------  -------
----------------  -----------------
 2    +3.531482E+00  +9.712436E-05   +2.232753E-01    +5.610633E+03      0
+2.757554E+03      +4.666262E+04
================================================================================
====================================

Cumulative Statistics:
==============================================================================
Computation  Optimizer  Optimization   Objective   Fixed Point  Contraction
   Time      Converged   Iterations   Evaluations  Iterations   Evaluations
-----------  ---------  ------------  -----------  -----------  -----------
 00:03:58       Yes          93           108        566451       1737636
==============================================================================

Nonlinear Coefficient Estimates (Unadjusted SEs in Parentheses):
==========================================
 Sigma:       satellite          wired
---------  ---------------  ---------------
satellite   +2.362757E+00
           (+1.164699E+00)

  wired     +0.000000E+00    +2.132738E+00
                            (+1.145847E+00)
==========================================

Beta Estimates (Unadjusted SEs in Parentheses):
==================================================================
       x            satellite          wired           prices
---------------  ---------------  ---------------  ---------------
 +1.322723E+00    +6.018131E+00    +6.082889E+00    -3.017711E+00
(+2.864467E-01)  (+1.770085E+00)  (+1.781485E+00)  (+7.438821E-01)
==================================================================

Gamma Estimates (Unadjusted SEs in Parentheses):
================================
       1                w
---------------  ---------------
 +1.865217E+00    +4.711512E-01
(+1.095320E-01)  (+6.369255E-02)
================================
```

In [13]:

```
opt_iv = res_joint.compute_optimal_instruments(method='approximate')
problem_opt = opt_iv.to_problem()

res_opt = problem_opt.solve(
    sigma=res_joint.sigma,
    beta=res_joint.beta,
    gamma=res_joint.gamma,
    optimization=pyblp.Optimization('l-bfgs-b', {'gtol': 1e-8}),
    iteration=pyblp.Iteration('squarem'),
    W_type='unadjusted',
    se_type='unadjusted',
```

```
)
print(res_opt)
```

```
Problem Results Summary:
================================================================================
====================================
GMM     Objective      Projected    Reduced Hessian  Reduced Hessian  Clipped
Weighting Matrix  Covariance Matrix
Step      Value      Gradient Norm  Min Eigenvalue   Max Eigenvalue   Shares
Condition Number  Condition Number
----  -------------  -------------  ---------------  ---------------  -------
----------------  -----------------
 2    +8.175156E+00  +6.110306E-04   +1.514734E-01    +1.147218E+04      0
+1.097475E+20      +4.638752E+04
================================================================================
====================================

Cumulative Statistics:
==============================================================================
Computation  Optimizer  Optimization   Objective   Fixed Point  Contraction
   Time      Converged   Iterations   Evaluations  Iterations   Evaluations
-----------  ---------  ------------  -----------  -----------  -----------
 00:05:52       No           85           160        802565       2470818
==============================================================================

Nonlinear Coefficient Estimates (Unadjusted SEs in Parentheses):
============================================
 Sigma:       satellite          wired
---------  ---------------  ---------------
satellite   +9.957701E-01
           (+1.038684E+00)

  wired     +0.000000E+00    +3.387350E-01
                            (+2.351189E+00)
============================================

Beta Estimates (Unadjusted SEs in Parentheses):
================================================================
       x            satellite          wired           prices
---------------  ---------------  ---------------  ---------------
 +1.070620E+00    +4.709703E+00    +4.799504E+00    -2.397605E+00
(+2.251353E-01)  (+1.453674E+00)  (+1.468504E+00)  (+5.961967E-01)
================================================================

Gamma Estimates (Unadjusted SEs in Parentheses):
================================
       1                w
---------------  ---------------
 +1.882818E+00    +4.331421E-01
(+1.373932E-01)  (+6.584693E-02)
================================
```

In [14]:

```
def get_params(res):
    b = res.beta.flatten()
    bse = res.beta_se.flatten()
    s = np.diag(res.sigma)
    sse = np.diag(res.sigma_se)
    return np.r_[b, s].round(4), np.r_[bse, sse].round(4)

vd, sd = get_params(res_demand)
```

```
vj, sj = get_params(res_joint)
vo, so = get_params(res_opt)

pd.DataFrame({
    'param':        ['beta1','beta_sat','beta_wire','alpha','sigma_sat','sigma_
    'true':         [1.0,'~4','~4',-2.0,1.0,1.0],
    'demand_est':   vd,  'demand_se': sd,
    'joint_est':    vj,  'joint_se':  sj,
    'optiv_est':    vo,  'optiv_se':  so
})
```

Out[14]:

	param	true	demand_est	demand_se	joint_est	joint_se	optiv_est	optiv_se
0	beta1	1.0	1.3104	0.6612	1.3227	0.2864	1.0706	0.2251
1	beta_sat	~4	6.1155	4.1969	6.0181	1.7701	4.7097	1.4537
2	beta_wire	~4	5.1881	2.3345	6.0829	1.7815	4.7995	1.4685
3	alpha	-2.0	-2.9203	1.6850	-3.0177	0.7439	-2.3976	0.5962
4	sigma_sat	1.0	0.2821	1.9261	2.3628	1.1647	0.9958	1.0387
5	sigma_wire	1.0	3.4025	5.6798	2.1327	1.1458	0.3387	2.3512

Q6: BLP Elasticities and Diversion Ratios

In [15]:

```
res_pref = res_joint

blp_elas = res_pref.compute_elasticities().reshape(T, J, J).mean(axis=0)
blp_div = res_pref.compute_diversion_ratios().reshape(T, J, J).mean(axis=0)

print('BLP elasticity matrix:')
display(matrix_df(blp_elas))

print('\nBLP diversion ratios:')
display(matrix_df(blp_div))

print('\nTrue diversion ratios:')
display(matrix_df(true_div))

print('\nOwn-price comparison:')
display(pd.DataFrame({
    'product': ['J1','J2','J3','J4'],
    'blp_est': np.diag(blp_elas).round(4),
    'true':    true_own_elas.round(4),
    'diff':    (np.diag(blp_elas) - true_own_elas).round(4)
}))
```

BLP elasticity matrix:

	J1	J2	J3	J4
J1	-5.4710	2.5465	0.6410	0.6607
J2	2.5134	-5.6896	0.6410	0.6607
J3	0.6350	0.6621	-5.5943	2.4413
J4	0.6350	0.6621	2.4311	-5.6083

```
BLP diversion ratios:
```

	J1	J2	J3	J4
J1	0.3129	0.4150	0.1323	0.1397
J2	0.4228	0.3154	0.1309	0.1309
J3	0.1298	0.1292	0.3314	0.4096
J4	0.1316	0.1227	0.4107	0.3349

```
True diversion ratios:
```

	J1	J2	J3	J4
J1	0.3338	0.2170	0.2204	0.2288
J2	0.2194	0.3370	0.2216	0.2220
J3	0.2193	0.2185	0.3350	0.2272
J4	0.2246	0.2150	0.2232	0.3371

```
Own-price comparison:
```

	product	blp_est	true	diff
0	J1	-5.4710	-4.2500	-1.2210
1	J2	-5.6896	-4.3714	-1.3182
2	J3	-5.5943	-4.2711	-1.3232
3	J4	-5.6083	-4.2576	-1.3507

Q9

In [16]:

```
costs = res_pref.compute_costs()
pre_prices = res_pref.compute_prices()
avg_pre = pre_prices.reshape(T, J).mean(axis=0)

firm_ids_m12 = pd_blp['firm_ids'].values.copy()
firm_ids_m12[firm_ids_m12 == 1] = 0  # firms 1+2 merge

post_m12 = res_pref.compute_prices(firm_ids=firm_ids_m12, costs=costs)
delta_m12 = (post_m12 - pre_prices).reshape(T, J).mean(axis=0)

pd.DataFrame({
    'product':    ['J1 (sat)','J2 (sat)','J3 (wire)','J4 (wire)'],
    'pre':        avg_pre.round(4),
    'post':       post_m12.reshape(T,J).mean(axis=0).round(4),
    'delta_p':    delta_m12.round(4)
})
```

Out[16]:

	product	pre	post	delta_p
0	J1 (sat)	2.6458	3.0041	0.3583
1	J2 (sat)	2.7293	3.0940	0.3647
2	J3 (wire)	2.6594	2.6667	0.0073
3	J4 (wire)	2.6675	2.6734	0.0059

Q10: Merger of Firms 1 and 3

In [17]:

```
firm_ids_m13 = pd_blp['firm_ids'].values.copy()
firm_ids_m13[firm_ids_m13 == 2] = 0  # firms 1+3 merge

post_m13 = res_pref.compute_prices(firm_ids=firm_ids_m13, costs=costs)
delta_m13 = (post_m13 - pre_prices).reshape(T, J).mean(axis=0)

pd.DataFrame({
    'product':   ['J1 (sat)','J2 (sat)','J3 (wire)','J4 (wire)'],
    'dp_m12':    delta_m12.round(4),
    'dp_m13':    delta_m13.round(4)
})
```

Out[17]:

	product	dp_m12	dp_m13
0	J1 (sat)	0.3583	0.0797
1	J2 (sat)	0.3647	0.0093
2	J3 (wire)	0.0073	0.0828
3	J4 (wire)	0.0059	0.0083

Q12: Merger 1+2 with 15% Cost Reduction

In [18]:

```
merger_costs = costs.copy()
merger_costs[pd_blp['firm_ids'].isin([0,1]).values] *= 0.85

post_eff = res_pref.compute_prices(firm_ids=firm_ids_m12, costs=merger_costs)
delta_eff = (post_eff - pre_prices).reshape(T, J).mean(axis=0)

pd.DataFrame({
    'product':      ['J1 (sat)','J2 (sat)','J3 (wire)','J4 (wire)'],
    'dp_no_eff':    delta_m12.round(4),
    'dp_15pct_eff': delta_eff.round(4)
})
```

Out[18]:

	product	dp_no_eff	dp_15pct_eff
0	J1 (sat)	0.3583	0.1498
1	J2 (sat)	0.3647	0.1428
2	J3 (wire)	0.0073	0.0018
3	J4 (wire)	0.0059	0.0005

In [21]:
```
cs_pre = res_pref.compute_consumer_surpluses()

cs_post_no_eff = res_pref.compute_consumer_surpluses(prices=post_m12)

cs_post_eff = res_pref.compute_consumer_surpluses(prices=post_eff)

# Changes in consumer surplus
dcs_no_eff = (cs_post_no_eff - cs_pre).sum()
dcs_eff = (cs_post_eff - cs_pre).sum()

avg_dcs_no_eff = dcs_no_eff / T
avg_dcs_eff = dcs_eff / T

cs_table = pd.DataFrame({
    'Scenario': ['Merger without efficiencies', 'Merger with 15% efficiencies']
    'Total ΔCS (Mt = 1)': [round(dcs_no_eff, 4), round(dcs_eff, 4)],
    'Average ΔCS per market': [round(avg_dcs_no_eff, 4), round(avg_dcs_eff, 4)]
})

display(cs_table)
```

	Scenario	Total ΔCS (Mt = 1)	Average ΔCS per market
0	Merger without efficiencies	-51.3878	-0.0856
1	Merger with 15% efficiencies	-19.8730	-0.0331